Web Engineering & Development (SWE 363)

Back-end Development Fundamentals

In today's lecture:

- Express.js
  - Web Servers
  - Routes
  - Middleware

Reference:

- Zybook: 6.4

6.4 Express.js

What is Express.js?

> **Express.js** is a web framework for Node.js

Express simplifies:

- Creating web servers
- Handling HTTP requests and responses
- Defining routes (URLs)
- Using middleware

Think of Express as a toolkit that makes building web servers much easier!

Why Use Express.js?

Without Express:

- Write lots of code
- Handle HTTP manually
- Complex routing logic
- More time-consuming

With Express:

- Simple, clean code
- Built-in HTTP handling
- Easy route definition
- Fast development

Installing Express

Create a new project:

```
npm init -y
```

Install Express:

```
npm install express
```

Install Cross-Origin Resource Sharing (CORS) (for frontend communication):

```
npm install cors
```

Creating Your First Express Server

Basic server setup:

```
import express from "express";

const app = express();
const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Run the server:

```
node server.js
```

What is a Web Server?

> A **web server** listens for requests and sends responses

Simple analogy:

- **Restaurant:** Server listens for orders (requests)
- **Kitchen:** Server processes the order
- **Waiter:** Server brings food back (response)

In web development:

- Browser sends a **request** (asks for data)
- Server processes the request
- Server sends a **response** (returns data)

Understanding Requests and Responses

Request (req)

- Data **from** the client
- URL, method (GET/POST)
- Headers, body
- Query parameters

Response (res)

- Data **to** the client
- Status code (200, 404, etc.)
- JSON, HTML, text
- Headers

What are Routes?

> **Routes** define what happens when someone visits a specific URL

Example:

- `/` → Home page
- `/api/quote` → Get a random quote
- `/api/users` → Get user data

Routes organize your API - each URL does something different!

Why Do We Need Routes?

- **Organization:** Each URL has a specific purpose
- **Clarity:** Easy to understand what each endpoint does
- **Separation:** Different features in different routes
- **Scalability:** Easy to add new features

Example:

```
/api/students  → Student data
/api/teachers  → Teacher data
/api/courses   → Course data
```

Creating Your First Route

Basic GET route:

```
app.get("/", (req, res) => {
  res.send("Welcome to my API!");
});
```

What this does:

- When someone visits `http://localhost:3000/`
- Server responds with: "Welcome to my API!"

Route Syntax

General format:

```
app.METHOD(path, handler)
```

Components:

- `app` → Express application
- `METHOD` → HTTP method (get, post, put, delete)
- `path` → URL path (e.g., "/api/quote")
- `handler` → Function that handles the request

HTTP Methods

Method	Purpose	Example
GET	Retrieve data	Get a quote
POST	Create data	Create a user
PUT	Update data	Update user info
DELETE	Delete data	Delete a user

GET Route Example

Simple GET route:

```
app.get("/api/quote", (req, res) => {
  const quote = "Talk is cheap. Show me the code.";
  res.json({ quote: quote });
});
```

Test it:

- Visit: `http://localhost:3000/api/quote`
- Response: `{ "quote": "Talk is cheap. Show me the code." }`

Sending Different Response Types

Send plain text:

```
res.send("Hello World");
```

Send JSON:

```
res.json({ message: "Success", data: [...] });
```

Send status code:

```
res.status(404).send("Not Found");
```

Multiple Routes Example

```
app.get("/", (req, res) => {
  res.send("Welcome to the Quote Generator API");
});

app.get("/api/quote", (req, res) => {
  const quote = getRandomQuote();
  res.json({ quote });
});

app.get("/api/about", (req, res) => {
  res.json({
    name: "Quote API",
    version: "1.0.0"
  });
});
```

What is Middleware?

> **Middleware** are functions that run between a request and a response

Think of middleware as:

- **Security guards** checking requests
- **Helpers** that process data before it reaches your route
- **Loggers** that record what's happening

Middleware Flow

```
Request → Middleware 1 → Middleware 2 → Route Handler → Response
```

Example:

1. Request arrives
2. CORS middleware checks origin
3. Logger middleware records request
4. Route handler processes request
5. Response sent back

Using Middleware

Basic middleware:

```
app.use((req, res, next) => {
  console.log("Request received:", req.url);
  next(); // Continue to next middleware/route
});
```

Important: Always call `next()` to continue!

CORS Middleware

> **CORS** (Cross-Origin Resource Sharing) allows frontend to communicate with backend

Why we need it:

- Frontend runs on `http://localhost:5173`
- Backend runs on `http://localhost:3000`
- Browsers block cross-origin requests by default

Solution:

```
import cors from "cors";
app.use(cors());
```

CORS Example

Without CORS:

```
Frontend (port 5173) → ❌ Blocked → Backend (port 3000)
```

With CORS:

```
import cors from "cors";
app.use(cors());
```

```
Frontend (port 5173) → ✅ Allowed → Backend (port 3000)
```

Built-in Express Middleware

Parse JSON data:

```
app.use(express.json());
```

Parse URL-encoded data:

```
app.use(express.urlencoded({ extended: true }));
```

These allow your server to read request body data!

Complete Server Example

```
import express from "express";
import cors from "cors";

const app = express();
const PORT = 3000;

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.get("/", (req, res) => {
  res.send("Welcome to the Quote Generator API");
});
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Request Object (req)

Common properties:

```
app.get("/api/user/:id", (req, res) => {
  req.params.id      // URL parameters
  req.query.search   // Query string (?search=hello)
  req.body           // Request body (POST data)
  req.headers        // Request headers
  req.url            // Request URL
});
```

Response Object (res)

Common methods:

```
res.send("Text")              // Send text
res.json({ data })            // Send JSON
res.status(404)               // Set status code
res.sendStatus(200)           // Send status + message
res.redirect("/")             // Redirect to URL
```

Route Parameters

Dynamic routes:

```
app.get("/api/user/:id", (req, res) => {
  const userId = req.params.id;
  res.json({ userId });
});
```

Example:

- URL: `/api/user/123`
- `req.params.id` = `"123"`

Query Parameters

Query strings:

```
app.get("/api/search", (req, res) => {
  const query = req.query.q;
  res.json({ search: query });
});
```

Example:

- URL: `/api/search?q=javascript`
- `req.query.q` = `"javascript"`

POST Route Example

Handle POST requests:

```
app.post("/api/users", (req, res) => {
  const newUser = req.body;
  // Save user to database
  res.status(201).json({
    message: "User created",
    user: newUser
  });
});
```

Note: Requires `app.use(express.json())` middleware!

Error Handling

Basic error handling:

```
app.get("/api/data", (req, res) => {
  try {
    // Your code here
    res.json({ data: "success" });
  } catch (error) {
    res.status(500).json({
      error: "Something went wrong"
    });
  }
});
```

package.json Review

What is package.json?

- Configuration file for your project
- Lists all dependencies
- Defines project metadata
- Contains scripts

```
{
  "name": "quote-api",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5" }
}
```

package-lock.json

What is package-lock.json?

- Records **exact versions** of all packages
- Ensures **consistent installs**
- **Auto-generated** - don't edit manually!

Why it matters:

- Team members get same versions
- Production deployments are predictable
- Prevents "works on my machine" issues

Summary Table

Concept	Purpose	Example
Express	Web framework	`const app = express()`
Route	Handle URL	`app.get("/", handler)`
Middleware	Process requests	`app.use(cors())`
req	Request data	`req.params , req.body`
res	Send response	`res.json() , res.send()`

30m

Demo

6.3 Express

- Set up Express server
- Create routes
- Use middleware (CORS)
- Connect frontend to backend